

CSP算法可视化工具

开发指南

从零到一的完整开发路线 - 面向个人开发者的实战手册

定位: 教学辅助工具 + 培训业务差异化引擎

技术栈: 纯HTML/CSS/JavaScript (AI辅助开发)

目标用户: CSP-J/S竞赛学生及培训教师

目录

一、项目概述与定位

二、核心交互体验设计

2.1 页面布局标准

2.2 交互功能规范

2.3 用户操作流程

三、技术架构与选型

3.1 整体架构图

3.2 技术选型方案

3.3 文件目录结构

四、模板体系设计

4.1 三层模板架构

4.2 公共CSS规范

4.3 公共JS模块

五、算法内容规划

5.1 MVP优先级分层

5.2 每个算法的交付标准

六、AI辅助开发流程

6.1 开发流程总览

6.2 AI生成规范与Prompt模板

6.3 质量审核清单

七、分步开发路线图

八、部署与发布策略

九、长期演进方向

一、项目概述与定位

1.1 项目一句话定义

一款面向CSP-J/S信息学竞赛的算法可视化教学工具,帮助学生直观理解算法执行过程,降低算法学习的认知门槛。同时作为培训教师的教学辅助工具,提升课堂效果和学生续费率。

1.2 核心价值主张

- 可自定义数据: 学生可以输入自己的测试数据,观察算法在不同输入下的执行过程
- 步骤级交互控制: 播放/暂停/单步前进/单步后退/速度调节,学生掌控学习节奏
- 代码同步高亮: 算法源代码与可视化动画行级同步,代码执行到哪一行,可视化就演示到哪一步
- 中文友好: 全中文界面,贴合CSP考纲,填补英文工具在国内竞赛培训中的空白

1.3 产品定位三角形

定位维度	具体内容	价值
教学辅助	在课堂上配合CSP课程使用	提升教学效果和学生理解
引流工具	在线上(B站/小红书)发布可视化视频	降低招生获客成本
差异化壁垒	有自己开发的教学工具的老师	提升续费率和议价能力

1.4 MVP目标

第一阶段目标: 完成15-20个高频CSP算法的可视化页面,覆盖排序、搜索、基础数据结构、入门级DP,形成可对外展示的最小产品。

第二阶段目标: 在MVP基础上扩展到25-30个算法,增加图论、贪心等专题,并引入CSP真题关联功能。

二、核心交互体验设计

2.1 页面布局标准

每个算法页面采用统一的三栏布局:

区域	位置	内容	宽度占比
代码面板	左侧	算法源代码(带行号,支持行级高亮)	25%
可视化画布	中间	算法执行过程的可视化动画	50%

区域	位置	内容	宽度占比
控制面板	右侧	操作按钮 + 参数设置 + 状态信息	25%

顶部区域: 算法名称 + 简短描述 + 难度标签(入门/基础/进阶) + 返回主页按钮

底部区域: 算法原理简述 + 时间/空间复杂度 + CSP相关真题链接(可选)

2.2 交互功能规范

功能	描述	优先级
播放/暂停	控制动画的播放与暂停	P0(必做)
单步前进	执行下一步操作,便于逐细节理解	P0(必做)
单步后退	回退到上一步操作,便于反复查看	P0(必做)
速度调节	0.5x / 1x / 2x / 5x 四档	P0(必做)
重置	恢复到初始状态,重新运行	P0(必做)
自定义数据	用户输入自己的测试数据	P0(必做)
随机生成	随机生成一组测试数据	P1(建议做)
代码同步高亮	当前执行步骤对应的代码行高亮	P0(必做)
步骤说明文字	每步操作的文字解释	P0(必做)

2.3 用户操作流程

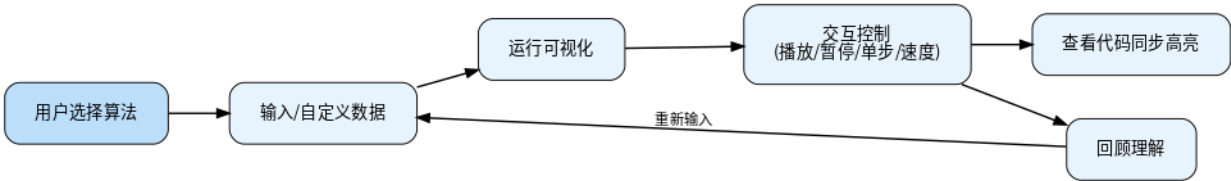


图1: 用户操作流程

三、技术架构与选型

3.1 整体架构图

项目采用分层架构: 公共资源层提供全局样式和工具函数,算法页面层是每个算法的独立HTML文件,通过主页index.html统一导航。

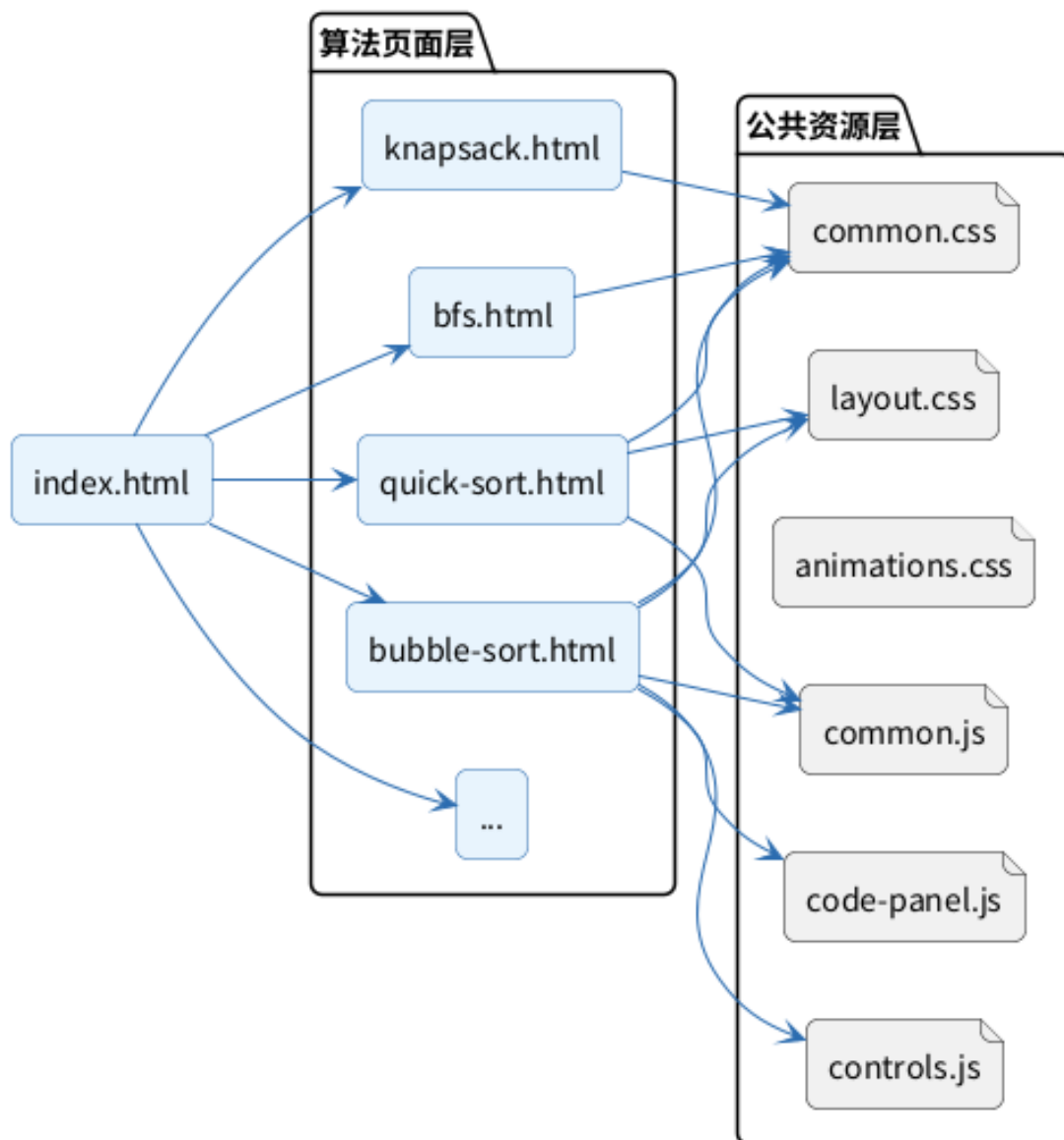


图2: 项目架构图

3.2 技术选型方案

层面	选型	理由
样式框架	Tailwind CSS	写样式极快, AI生成效率高, 无需手写大量CSS
动画方案	CSS Transition + requestAnimationFrame	轻量够用, 不需要额外动画库
代码高亮	Prism.js / Highlight.js	一行引入即用, 支持C++语法高亮
图形绘制	Canvas(复杂) / DOM操作(简单)	不引入D3.js等重库, 降低复杂度
版本管理	Git + GitHub	免费托管, GitHub Pages零成本部署
前端框架	纯HTML/CSS/JS (暂不引入框架)	降低学习成本, AI生成单个HTML文件效率最高

为什么不选Vue/React?

初期用纯HTML文件的最大好处是AI生成效率极高。每个算法是一个完整的独立单元,AI不需要理解框架的组件体系、路由、状态管理等概念。后期如果维护成本上升(算法数量超过20个),可以考虑迁移到Vue。迁移成本可控,因为公共部分已经被抽离成独立模块。

3.3 文件目录结构

从第一天起就用清晰的目录结构,为后期扩展打好基础:

```
project/
  index.html -- 主页,算法分类导航
  css/
    common.css -- 全局样式:配色方案、字体定义、通用类
    layout.css -- 布局模板:三栏布局、响应式断点
    animations.css -- 动画通用样式:过渡效果、高亮动画
  js/
    common.js -- 工具函数:步骤控制器、速度管理、事件绑定
    code-panel.js -- 代码面板组件:行号渲染、行级高亮
    controls.js -- 控制栏组件:播放/暂停/单步/速度按钮
  algorithms/
    sort/
      bubble-sort.html
      quick-sort.html
      merge-sort.html
    search/
      bfs.html
      dfs.html
    data-structure/
      binary-tree.html
    dp/
      knapsack.html
    lcs.html
  assets/
    -- 图片、图标等静态资源
```

四、模板体系设计

4.1 三层模板架构

模板分三层设计,每一层约束下一层的实现方式。第一层和第二层需要精心手写打磨,第三层交给AI批量生成。

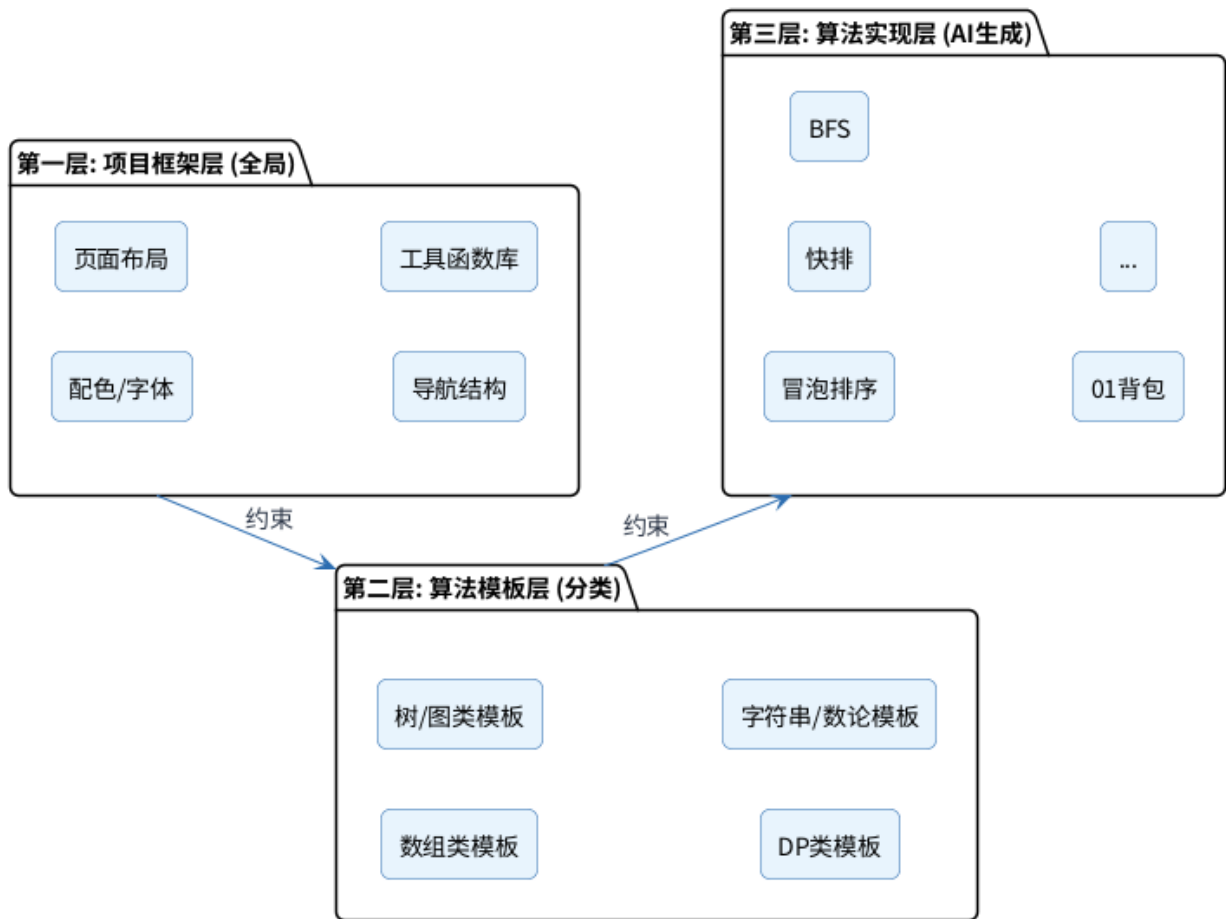


图3: 三层模板架构

第一层: 项目框架层 (全局)

这一层定义整个项目的视觉风格和基础交互范式,所有算法页面必须遵循:

- 统一配色方案: 主色调(深蓝)、辅助色(浅蓝)、高亮色(橙色/绿色)、背景色(白色/浅灰)
- 统一字体: 代码区使用等宽字体(Noto Sans Mono / Consolas),UI区使用CJK无衬线字体
- 统一布局: 左代码-中画布-右控制的三栏布局,顶部标题栏,底部信息栏
- 统一导航: 每个页面顶部有返回主页的面包屑导航
- 统一动画规范: 默认过渡时间300ms,高亮闪烁200ms,步骤切换动画500ms

第二层: 算法模板层 (分类)

不同类型算法的可视化主体不同,至少需要以下模板类型:

模板类型	适用算法	可视化主体	交互特征
数组类	排序、双指针、滑动窗口	一维数组条形图	元素比较/交换高亮
树/图类	DFS/BFS、最短路、MST	图结构(节点+边)	节点/边的遍历高亮
DP类	背包、LCS、区间DP	二维表格/矩阵	单元格填充动画

模板类型	适用算法	可视化主体	交互特征
字符串/数论	KMP、哈希、筛法	自定义可视化	模式匹配/标记动画

第三层: 算法实现层 (AI生成)

每个具体算法的实现,包括数据初始化、步骤生成逻辑、高亮规则、伪代码同步等。这一层交给AI批量生成,但必须遵循前两层定义的规范。

4.2 公共CSS规范

common.css 应该定义:

- CSS变量定义: --primary, --accent, --highlight, --bg, --text, --code-bg 等全局颜色变量
- 通用组件样式: 按钮(.btn, .btn-primary, .btn-secondary)、卡片(.card)、标签(.tag)
- 高亮状态类: .highlight-active, .highlight-compare, .highlight-swap, .highlight-visited, .highlight-current
- 响应式断点: 移动端(<768px)切换为上下布局,平板(768-1024px)两栏,桌面(>1024px)三栏

4.3 公共JS模块

common.js - 步骤控制器,是整个项目最核心的公共模块:

功能: 维护一个步骤数组(steps),每个步骤包含{data, highlight, codeLine, description}。提供next(), prev(), play(), pause(), setSpeed()方法。所有算法页面的交互控制都基于这个控制器。

code-panel.js - 代码面板组件: 接收代码字符串和当前高亮行号,渲染带行号的代码并同步高亮。

controls.js - 控制栏组件: 渲染播放/暂停/前进/后退/重置/速度按钮,绑定事件到步骤控制器。

五、算法内容规划

5.1 MVP优先级分层

按照CSP考频和教学重要性分为三个梯队。先做第一梯队,有余力再做第二梯队,第三梯队暂不启动。

第一梯队 (MVP,15-20个算法)

这些是CSP-J必考、CSP-S高频出现的算法,也是教学中学生最需要可视化辅助理解的内容:

分类	算法	CSP考频	难度
排序	冒泡排序	高	入门

分类	算法	CSP考频	难度
排序	选择排序	高	入门
排序	插入排序	高	入门
排序	快速排序	极高	基础
排序	归并排序	高	基础
排序	堆排序	中	基础
搜索	二分查找	极高	入门
搜索	深度优先搜索(DFS)	极高	基础
搜索	广度优先搜索(BFS)	极高	基础
数据结构	二叉树遍历(前/中/后/层)	高	基础
数据结构	栈和队列	高	入门
双指针	双指针/滑动窗口	高	基础
DP	0-1背包问题	高	基础
DP	最长递增子序列(LIS)	中	基础
DP	最长公共子序列(LCS)	中	基础

第二梯队 (扩展,8-12个算法)

分类	算法	CSP考频	难度
图论	Dijkstra最短路	高	进阶
图论	最小生成树(Kruskal/Prim)	中	进阶
图论	拓扑排序	中	基础
数据结构	并查集	高	基础
贪心	区间调度/活动选择	中	基础
贪心	哈夫曼编码	中	基础
DP	区间DP	中	进阶
DP	树形DP(基础)	中	进阶
搜索	迭代加深搜索	低	进阶

分类	算法	CSP考频	难度
字符串	KMP字符串匹配	中	进阶

第三梯队 (暂不启动)

高级数据结构(线段树、平衡树、LCT)、复杂数论(莫比乌斯反演、BSGS)、高级DP(数位DP、状压DP复杂应用)。这些算法你自己也需要深入学习后再考虑可视化。

5.2 每个算法的交付标准

每个算法页面必须达到以下标准才能发布:

检查项	具体要求
动画完整性	动画能从第一步完整执行到最后一步,不卡住、不跳步
交互控制	播放/暂停/单步前进/后退/速度调节全部可用
自定义数据	用户输入数据后能正确运行,处理边界情况(空输入、单元素、重复值)
代码同步	代码行高亮与当前执行步骤准确对应
步骤说明	每步操作有清晰的中文文字说明
响应式	在1024px以上宽度正常显示,768px以下可用(可接受简化布局)
可解释性	你自己能对着这个可视化页面,向学生讲清楚算法的每一步

六、AI辅助开发流程

6.1 开发流程总览

整个开发遵循以下流程:前半部分手写验证,后半部分AI批量生成。



图4: 开发流程总览

关键原则: 前3个算法全程手写或精调,摸清模板边界和AI出错模式。第4个开始逐步引入AI生成,但每次都要严格审核。

6.2 AI生成规范与Prompt模板

为了让AI高效生成符合你模板的代码,每次生成一个新算法时,需要提供以下上下文:

Prompt结构模板:

你是一个算法可视化页面开发者。请按照以下规范生成一个[算法名称]的可视化HTML页面。

1. 技术约束:

- 纯HTML/CSS/JS,单文件,引入Tailwind CSS CDN、Prism.js CDN

- 引入公共CSS: common.css, layout.css, animations.css

- 引入公共JS: common.js, code-panel.js, controls.js

2. 布局要求:

- 三栏布局: 左代码面板(25%)、中可视化画布(50%)、右控制面板(25%)

- 顶部: 算法名称 + 难度标签 + 返回按钮

- 底部: 算法原理简述 + 复杂度分析

3. 交互要求:

- 播放/暂停、单步前进/后退、速度调节(0.5x/1x/2x/5x)、重置

- 用户可自定义输入数据

- 代码行级同步高亮

- 每步操作有中文说明文字

4. 算法逻辑:

- [这里写算法的核心逻辑和步骤描述]

- 算法源代码: [粘贴C++代码]

5. 可视化类型:

- [数组类/树图类/DP类/字符串类]

- [描述具体的可视化方式]

6.3 质量审核清单

每个AI生成的算法页面,必须逐项通过以下审核:

序号	检查项	通过标准
1	动画完整性	从第一步到最后一步无卡顿、无跳步
2	速度控制	四档速度全部正常工作
3	自定义数据	空输入/单元素/重复值/边界值均正确处理
4	代码同步	高亮行号与实际执行步骤一一对应
5	步骤后退	后退操作能正确恢复状态(不是简单的重新开始)
6	重置功能	重置后数据恢复初始状态

序号	检查项	通过标准
7	手机适配	在手机浏览器上可查看(允许简化布局)
8	可解释性	你能用这个页面向学生讲清楚算法

审核不通过的处理: 不要直接让AI'修复',而是明确指出哪个检查项不通过、具体现象是什么,然后让AI针对该问题修改。模糊的反馈只会得到模糊的修复。

七、分步开发路线图

以下步骤无需严格按时间执行,按你的节奏来。每完成一步确认没问题再进入下一步。

第一步: 分析现有HTML文件

拿出你已经做好的可交互HTML文件,做以下分析:

- 画出它的页面结构图:头部/主体/侧边栏/底部各放了什么
- 列出它用到的CSS类名和JS函数,标记哪些是'每个算法都需要的'
- 标记哪些部分可以抽象为公共模块,哪些是算法特有的
- 记录它的交互逻辑:播放/暂停怎么控制的?步骤怎么存储的?高亮怎么触发的?

产出: 一份现有HTML的结构分析笔记(可以手写,不需要正式文档)。

第二步: 搭建项目框架

根据分析结果,创建项目目录结构,编写公共CSS和公共JS模块:

- 创建目录结构(参见第三章3.3节)
- 编写 common.css:定义CSS变量、通用组件样式、高亮状态类
- 编写 layout.css:定义三栏布局、响应式断点
- 编写 animations.css:定义通用动画(高亮闪烁、元素移动、颜色过渡)
- 编写 common.js:实现步骤控制器(核心!)
- 编写 code-panel.js 和 controls.js
- 创建 index.html:主页,列出所有算法的分类导航(此时大部分链接是占位)

产出: 一个可以打开的空壳主页,点击分类能看到占位内容。公共模块编写完成并通过简单测试。

第三步: 手写第一个算法(冒泡排序)

选择最简单的冒泡排序,全程手写,不依赖AI生成:

- 目的:验证公共模块是否好用,发现框架层的问题
- 用公共模块来构建页面,遇到不合理的地方就修改公共模块
- 这个算法不需要完美,但交互体验要达标(参见5.2节交付标准)
- 记录开发过程中遇到的问题和改进点

产出: 第一个完整的可交互算法页面。同时公共模块经过一轮实战打磨。

第四步: 手写第二、三个算法(快排 + 二分查找)

继续手写,覆盖不同模板类型(数组类 + 搜索类):

- 快排:验证数组类模板在更复杂算法中的表现
- 二分查找:验证非排序类算法的可视化方案
- 重点观察:公共模块是否需要进一步抽象?不同算法之间的重复代码有多少?
- 如果发现公共模块频繁需要修改,说明抽象不够好,回到第二步优化

产出: 三个手写算法页面,公共模块趋于稳定。可以形成一份"算法模板使用说明"。

第五步: 定义AI生成规范

基于前三个算法的开发经验,写一份AI生成规范文档:

- Prompt模板(参见第六章6.2节)
- 公共模块的使用方式(在Prompt中需要告诉AI引入哪些CSS/JS)
- 常见AI错误模式及修正方法(基于前三个算法的经验)
- 每类算法模板的可视化方式说明

产出: 一份AI生成Prompt模板 + 错误修正手册(自己用就行)。

第六步: AI批量生成第一梯队算法

用定义好的Prompt模板,让AI生成剩余的第一梯队算法:

- 每次生成一个,生成后立即审核(用6.3节的审核清单)
- 不通过的退回AI修改,通过的部署上线
- 每完成5个算法,回顾一下公共模块是否需要升级
- 优先完成排序类和搜索类(模板最成熟,AI生成质量最高)

产出: 15-20个第一梯队算法页面全部上线。

第七步: 教学实测与迭代

把工具带到你的CSP课堂上使用:

- 观察学生使用时的反应:哪些算法的展示效果好?哪些学生还是看不懂?
- 记录学生的反馈和问题
- 针对反馈修改对应的算法页面(优先修改高频使用的)
- 收集家长反馈,了解工具对续费意愿的影响

产出: 经过真实教学验证的第一版产品,有明确的改进方向清单。

八、部署与发布策略

8.1 零成本部署方案: GitHub Pages

前期完全不需要花钱部署。把项目推送到GitHub,开启GitHub Pages即可:

1. 在GitHub创建仓库(如 csp-algorithm-visualizer)
2. 将项目代码推送上去
3. 在仓库Settings > Pages中选择main分支,根目录
4. 几分钟后即可通过 <https://yourname.github.io/csp-algorithm-visualizer/> 访问
5. 每次git push,网站自动更新

8.2 后期升级路径

阶段	部署方案	成本	触发条件
MVP阶段	GitHub Pages	免费	算法数<20,无用户系统
成长阶段	云服务器(Vercel/阿里云)	0-100元/月	有稳定流量,需要自定义域名
商业化阶段	云服务器+数据库+后端	200-500元/月	需要用户系统/付费功能

九、长期演进方向

以下方向不需要现在考虑,但可以在开发过程中保持意识,为未来留好扩展空间:

方向	具体内容	预估周期
算法扩展	完成第二梯队算法,覆盖图论/贪心/DP进阶	MVP完成后3-6个月
真题关联	每个算法关联对应的CSP历年真题,提供'真题练习'入口	有基础算法量后

方向	具体内容	预估周期
用户系统	引入后端(Spring Boot),支持用户注册/登录/学习进度跟踪	需要商业化时
付费功能	基础算法免费,高级算法/真题解析付费解锁	有稳定流量后
内容生态	B站/小红书发布可视化讲解视频,SEO优化官网引流	MVP完成后即可启动
机构授权	将工具年费授权给其他CSP培训机构使用	工具有口碑后
春考/成人培训	利用中职教资+Java背景,拓展到春考计算机专业课培训	取得教资后

最后的建议: 不要试图一次性把所有事情做完。先把第一个冒泡排序做出来,放到网上,让一个学生试用。一个小小的里程碑,比完美的计划更有价值。